

Appraisal Page — Frontend Calculation Spec

Derived from `AppraisalPage.jsff` and `HRAAppraisalBean.java` (Oracle ADF legacy)

The screen has three groups of numbers

- Current column** — read-only. Comes from the server. Basic, JobAllowance, FixedOT, HRA, Transport, Gross.
- Proposed column** — the user types here. New Basic, New JobAllowance, New FixedOT, New HRA, New Transport. New Gross is read-only and calculated.
- Right side** — Net Increment, Increment %, Bonus, Bonus %. All four are typed by the user, and all four also get filled in automatically depending on what else the user touched.

Which fields cause a calculation

Nine fields. Nothing else does anything.

Field	Triggers?	What runs
New Basic, New JobAllowance, New FixedOT, New HRA, New Transport	Yes	Case 1 — full gross recalc
Bonus	Yes	Case 1 — same routine
Increment %	Yes	Case 2 — percent to components
Bonus %	Yes	Case 3 — bonus only
Net Increment	Yes	Case 4 — allocate, then Case 1
Ticket Period, Ticket Count, New Designation	No	Saved only, no math
Effective Date (Period From)	No	But it feeds the Arrears formula — see gotchas
Basic Salary %, Fixed Allowance % (Batch tab)	No	Settings read by Case 4

Calculations run when the user **leaves the box or presses Enter**. Not while typing. This matches the original ADF behaviour.

Before any calculation

- If FixedOT, HRA or Transport in the Proposed column is empty, copy the Current value into it. Only when it is empty — never overwrite something the user typed, and never if it is zero.
- Do **not** do this for Basic or JobAllowance.

```
if (New FixedOT is empty) New FixedOT = Current FixedOT
if (New HRA is empty) New HRA = Current HRA
if (New Transport is empty) New Transport = Current Transport
```

Case 1 — user edits any of the five salary boxes, or Bonus

Add up both columns:

```
Current Gross = Current (Basic + JobAllowance + FixedOT + Transport + HRA)
New Gross     = New     (Basic + JobAllowance + FixedOT + Transport + HRA)
```

Then:

```
Net Increment = New Gross - Current Gross

Increment %   = Net Increment / Current Gross * 100

Bonus %       = Bonus / (Current Basic + Current JobAllowance) * 100
                (skip if Bonus is zero)

Arrears       = Net Increment * months from Effective Date to today
                (only if Effective Date is in the past)
```

Bonus % uses the **Current** Basic and JobAllowance, not the new ones. That is how the old system works. Leave it as is.

Months are counted by calendar month boundaries, ignoring the day. So 31 Jan to 1 Feb counts as one month.

Update on screen: New Gross, Net Increment, Increment %, Bonus %, the row in the table below, and the totals at the bottom.

Case 2 — user types Increment %

Apply the percent to **Basic and JobAllowance only**. FixedOT, HRA and Transport stay as they are.

```
New Basic      = Current Basic      + (Current Basic      * % / 100)
New JobAllowance = Current JobAllowance + (Current JobAllowance * % / 100)
```

Then recalculate New Gross, Net Increment and Arrears exactly as in Case 1.

Do not touch Bonus or Bonus %. Do not overwrite the Increment % the user just typed.

Update on screen: New Basic, New JobAllowance, New Gross, Net Increment, table row, totals.

Case 3 — user types Bonus %

Just one line. Nothing else changes.

```
Bonus = (Current Basic + Current JobAllowance) * Bonus % / 100
```

Update on screen: Bonus, and the Bonus total at the bottom. Nothing else.

Case 4 — user types Net Increment

This one works backwards. The user says "give this person 500" and the system decides how much goes to Basic and how much to JobAllowance.

The split comes from two settings on the **Batch Update** tab: Basic Salary % and Fixed Allowance %.

First check three things. If any fails, show the message and stop — do not change any numbers.

- Basic Salary % is filled in
- Fixed Allowance % is filled in
- The two add up to exactly 100

Then:

```
if (amount > 0 AND employee HAS a JobAllowance)
    Basic goes up by      (amount * Basic % / 100)
    JobAllowance goes up by (whatever is left over)

if (amount > 0 AND employee has NO JobAllowance)
    all of it goes to Basic

if (amount <= 0)
    reset New Basic and New JobAllowance back to the Current values
    (the old system has no way to reduce a salary here)
```

After that, run Case 1's calculation. Net Increment will come back to the same number the user typed.

Fixed Allowance % is only used in the "adds up to 100" check. The actual JobAllowance amount is the leftover, so all rounding drift lands in JobAllowance.

Update on screen: New Basic, New JobAllowance, plus everything from Case 1.

Totals at the bottom

Recalculate these after every single change:

Total Gross Current	= sum of all Current Gross
Total Gross New	= sum of all New Gross
Increase	= Total Gross New - Total Gross Current
Increase %	= Increase / Total Gross Current * 100
Bonus	= sum of all Bonus
Arrears	= sum of all Arrears

Colours

A Proposed box turns **light green** when its value is different from the Current value next to it. A table row turns light green when its status is "Updated". The Current Promotion Date box turns **yellow** when it is empty.

Four things that will bite you

1. Infinite loops

Net Increment is typed by the user, but it is also written back by the calculation. Same for Increment % and Bonus %. If your code watches the value instead of watching for user input, it will trigger itself over and over. Make sure your own writes do not fire the handler.

2. Divide by zero

Increment % divides by Current Gross. If an employee has all zeros, this crashes. The old code crashes too. Show blank or zero instead.

3. The old percentages are wrong

The legacy code rounds before multiplying by 100, so 5.67% shows up as 6.00%. Do it properly — multiply first, then round to two decimals. Warn whoever compares the old screen to the new one, because the numbers will not match.

4. The four CTC boxes have no formula

Monthly CTC and Yearly CTC, both Current and New, are shown on the screen but nothing on the front end calculates them. They come from the server. Leave them read-only until someone tells you where they are worked out.

Also worth knowing: Arrears and Status do not appear on the edit form at all — they are grid-only columns. The edit panel writes them, the grid displays them.

Source: ASGPayrollViewController — AppraisalPage.jsff, HRAppraisalBean.java · Batch update, salary-master push and arrears recalculation are handled by Oracle stored procedures and are out of scope for the frontend.